

Class In Python

تعریف ساده شی گرای (OOP)

برنامه‌نویسی شی‌گرا (Object-Oriented Programming) روشی برای طراحی برنامه‌هاست که همه‌چیز رو به شکل شیء (Object) در نظر می‌گیره. هر شیء شامل ویژگی‌ها (data) و رفتارها (method) هست.

مثلاً یک ماشین:

- ویژگی: رنگ، مدل، سرعت

- رفتار: حرکت، ترمز، بوق

در OOP ما این مفاهیم رو با استفاده از کلاس‌ها پیاده‌سازی می‌کنیم.



(Attribute) تعریف: ویژگی

تعلق دارند. این داده‌ها (object) در پایتون به داده‌هایی گفته می‌شود که به یک شیء **attribute** ویژگی یا همان معمولاً اطلاعاتی درباره‌ی شیء هستند.

تعریف ساده

ویژگی، مشخصه‌ای از یک شیء است که اطلاعاتی را درباره‌ی آن شیء نگهداری می‌کند.

(Method) تعریف: متد

در پایتون به تابعی گفته می‌شود که درون یک کلاس تعریف شده و مخصوص شیء است. این متدها **method** متد یا همان معمولاً رفتاری (عملیاتی) را برای شیء انجام می‌دهند.

تعریف ساده

متد، تابعی است که داخل کلاس تعریف می‌شود و می‌تواند به ویژگی‌های شیء دسترسی داشته باشد یا آن‌ها را تغییر دهد.

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color
```

در این مثال

- ویژگی `brand` و `color` هستند
- `self.brand = brand` یعنی این ویژگی به شیء `self` اختصاص داده شده است.

```
my_car = Car("Toyota", "Red")
print(my_car.brand) # خروجی: Toyota
print(my_car.color) # خروجی: Red
```

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color

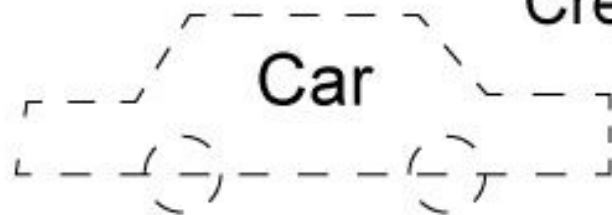
    def drive(self):
        print(f "The {self.color} {self.brand} is driving.")

my_car = Car("Toyota", "Red")
my_car.drive() # خروجی: The Red Toyota is driving.
```

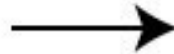
تمرین پیشنهادی برای دانش‌پذیران

- تعریف کنند **Book** یک کلاس
- **title, author, year**: سه ویژگی مثل
- که مشخصات کتاب را چاپ کند **describe()** یک متد

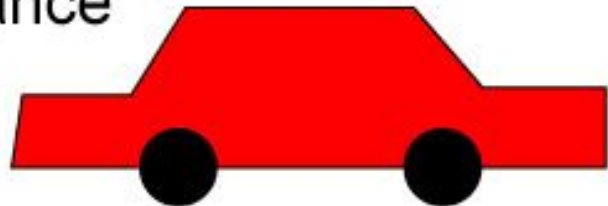
Class



Create an instance



Object



Properties	Methods - behaviors
color	start()
price	backward()
km	forward()
model	stop()

Property values	Methods
color: red	start()
price: 23,000	backward()
km: 1,200	forward()
model: Audi	stop()

برنامه‌نویسی ساخت‌یافته

```
name = "Ali"  
age = 30  
  
def say_hello():  
    print(f"Hello, {name}")
```

برنامه‌نویسی شی‌گرا

```
class Person:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
    def say_hello(self):  
        print(f"Hello, {self.name}")
```

و استفاده ازش

```
p = Person("Ali", 30)  
p.say_hello()
```

تعریف کلاس چیست؟

کلاس در پایتون مثل یک قالب یا نقشه‌ی اولیه برای ساخت اشیاء است. وقتی یک کلاس می‌سازید، در واقع دارید الگویی تعریف می‌کنید که بر اساس اون می‌شه اشیاء مختلف تولید کرد.

ساختار پایه‌ی یک کلاس:

```
class ClassName:  
    def __init__(self):  
        pass
```

کلمه کلیدی برای تعریف کلاس → **class**

CamelCase اسم کلاس با حرف بزرگ شروع می‌شه، مثل → **ClassName**

متدی که هنگام ساخت شیء خودکار اجرا می‌شه → **__init__**

اشاره به همان شیء‌ای که ساخته شده → **self**

چرا از `self` استفاده می‌کنیم؟

کلمه‌ی `self` در متدها برای اشاره به همان شیء‌ای است که متد روی آن صدا زده شده. مثلاً:

```
dog1 = Dog("Max", 3)
dog2 = Dog("Bella", 5)
```

- `self.name = name` یعنی ویژگی `name` متعلق به `dog1` است.

- `self` در واقع جایگزین `dog1` درون کلاس است.

ساخت چند شیء از یک کلاس

کلاس مثل قالب شیرینی‌پزی عمل می‌کند. شما می‌تونید چند تا شیء مختلف بسازید:

```
dog1 = Dog("Max", 3)

dog2 = Dog("Bella", 5)

dog1.bark() # Max says Woof!

dog2.bark() # Bella says Woof!
```

هر شیء ویژگی خودش رو داره. تغییر در `dog1` روی `dog2` تأثیری نداره.

متد `__init__` چیست؟

- متد `__init__` یک متد جادویی (special method) در پایتون است.
- هر وقت یک شیء جدید ساخته شود، این متد به طور خودکار اجرا می شود.
- هدف اصلی آن مقداردهی اولیه ویژگی های شیء است.

```
class MyClass:
```

```
    def __init__(self, param1, param2):
```

```
        self.param1 = param1
```

```
        self.param2 = param2
```

- ارامترهای داخل `__init__` هنگام ساخت شیء ارسال می شوند.
- `self.param1` یک ویژگی (attribute) جدید به شیء اضافه می کند و مقدار آن را تنظیم می کند.

تفاوت ویژگی‌های نمونه و ویژگی‌های کلاس

ویژگی‌های نمونه (Instance Attributes)

- به هر شیء به صورت جداگانه تعلق دارند.
- در متد `__init__` یا دیگر متدهای نمونه تعریف می‌شوند.
- مثال: `self.name = name`

ویژگی‌های کلاس (Class Attributes)

- مشترک بین همه اشیاء آن کلاس است.
- خارج از متدها و مستقیماً در بدنه کلاس تعریف می‌شوند.

```
class Dog:
    species = "Canis familiaris" # ویژگی کلاس
```

```
def __init__(self, name):
    self.name = name # ویژگی نمونه
```

```
dog1 = Dog("Max")
dog2 = Dog("Bella")
```

```
print(dog1.species) # Canis familiaris
print(dog2.species) # Canis familiaris
```

```
print(dog1.name)    # Max
print(dog2.name)    # Bella
```

```
Dog.species = "Canine" # تغییر ویژگی کلاس
```

```
print(dog1.species) # Canine
print(dog2.species) # Canine
```

```
class Person:
    def __init__(self, name):
        self.name = name

    def change_name(self, new_name):
        self.name = new_name

    def greet(self):
        print(f"Hello, my name is {self.name}")

p1 = Person("Sara")
p1.greet()      # Hello, my name is Sara
p1.change_name("Mina")
p1.greet()      # Hello, my name is Mina
```

خصوصی (private) یعنی چی؟

در پایتون، به صورت پیش فرض، تمام ویژگی‌ها و متدهای کلاس عمومی (public) هستند؛ یعنی از بیرون کلاس می‌توان به آن‌ها دسترسی داشت.

اما گاهی می‌خواهیم برخی ویژگی‌ها یا متدها فقط داخل کلاس قابل دسترس باشند و از بیرون مخفی باشند تا دستکاری ناخواسته جلوگیری شود. این‌ها را خصوصی می‌نامیم.

چگونه ویژگی و متد خصوصی بسازیم؟

در پایتون برای خصوصی کردن، نام ویژگی یا متد را با دو آندرلاین (underscore) شروع می‌کنیم:

دسترسی به ویژگی و متد خصوصی

class Person:

```
def __init__(self, name, age):
```

```
    self.__name = name    # ویژگی خصوصی
```

```
    self.__age = age
```

- داخل کلاس، می‌توان به ویژگی‌ها و متدهای خصوصی دسترسی داشت.

- از خارج کلاس، نمی‌توان مستقیم به آن‌ها دسترسی داشت.

```
def __secret_method(self):    # متد خصوصی
```

```
    print("This is a secret method!")
```

- ویژگی‌ها و متدهای خصوصی با پیشوند دو آندرلاین ساخته می‌شوند.

```
def public_method(self):
```

```
    print(f"My name is {self.__name}")
```

```
    self.__secret_method()
```

- این خصوصی بودن واقعی نیست ولی برای محافظت نسبی کاربرد دارد.

- دسترسی به خصوصی‌ها از خارج کلاس محدود می‌شود.

```
p = Person("Ali", 30)
```

```
p.public_method()    # درست است و کار می‌کند
```

```
# p.__name            # خطا! نمی‌توان به ویژگی خصوصی دسترسی داشت
```

```
# p.__secret_method() # خطا! نمی‌توان متد خصوصی را صدا زد
```

- کپسوله‌سازی کمک می‌کند برنامه‌ای منظم‌تر و ایمن‌تر بنویسیم.

نکته مهم: Name Mangling

پایتون با خصوصی کردن با دو آندرلاین، در واقع نام ویژگی را تغییر می‌دهد تا از دسترسی مستقیم جلوگیری کند. مثلاً `self.__name` به شکل `Person__name_` ذخیره می‌شود و می‌توان با این نام به آن دسترسی داشت (ولی توصیه نمی‌شود).

مثال دسترسی غیرمستقیم (غیرمعمول ولی ممکن)

```
print(p._Person__name)    # خروجی: Ali  
p._Person__secret_method() # اجرای متد خصوصی
```

چرا از خصوصی کردن استفاده کنیم؟

- حفاظت داده‌ها: جلوگیری از تغییر یا دسترسی ناخواسته به داده‌های حساس
- کپسوله‌سازی: مخفی کردن جزئیات داخلی کلاس برای ساده‌تر کردن استفاده
- کاهش خطا: محدود کردن دسترسی به بخش‌هایی که نباید توسط بخش‌های دیگر برنامه تغییر کنند

تمرین

یک کلاس **Account** تعریف کن که:

- داشته باشد (مقدار اولیه صفر) **__balance** ویژگی خصوصی
- برای افزودن مبلغ به موجودی باشد **deposit(amount)** متد
- که در صورت کافی بودن موجودی مبلغ را کم کند وگرنه پیام خطا دهد **withdraw(amount)** متد
- متد **get_balance()** که مقدار موجودی را برگرداند (بدون دسترسی مستقیم به ویژگی خصوصی)

وراثت یعنی چه؟

وراثت به ما اجازه می‌دهد کلاسی جدید (فرزند) بسازیم که رفتار و ویژگی‌های کلاس دیگر (والد) را به ارث ببرد.

مثلاً اگر کلاسی به نام **Animal** داریم، می‌توانیم از آن کلاس‌هایی مثل **Dog** یا **Cat** بسازیم که ویژگی‌های **Animal** را داشته باشند ولی چیزهایی هم به آن اضافه کنند.

```
class Parent:
```

```
    # ویژگی‌ها و متدهای والد
```

```
    ...
```

```
class Child(Parent):
```

```
    # ویژگی‌ها و متدهای جدید یا بازنویسی شده
```

```
    ...
```

super () چیه؟

super () یک تابع داخلی در پایتونه که به ما اجازه می‌ده به متدها یا سازنده (constructor) کلاس والد دسترسی پیدا کنیم، بدون اینکه اسم اون کلاس رو به‌طور مستقیم بنویسیم.

این یعنی وقتی از وراثت استفاده می‌کنیم و می‌خوایم بعضی از رفتارهای والد رو حفظ کنیم یا گسترش بدیم، **super ()** به کارمون میاد.

کاربرد	توضیح
فراخوانی <code>__init__</code> کلاس والد	برای مقداردهی ویژگی‌های ارث‌برده‌شده
استفاده از متدهای والد	وقتی متد رو <code>override</code> کردیم ولی هنوز می‌خوایم از نسخه والد استفاده کنیم
گسترش متدهای والد بدون تکرار کد	مثلاً ابتدا کد والد رو اجرا کنیم بعد چیزهایی بهش اضافه کنیم

چرا بهتره از `super()` استفاده کنیم؟

```
class Animal:
    def speak(self):
        print("Animal sound")
```

```
class Dog(Animal):
    def speak(self):
        super().speak() # اجرای متد والد
        print("Dog barks")
```

1. قابل‌توسعه‌تر بودن کد: اگر کلاس والد تغییر کنه، دیگه لازم نیست دستی همه جا اسم اون کلاس رو عوض کنیم.

2. پشتیبانی از وراثت چندگانه: وقتی چند کلاس به ارث برده شدن (Multiple Inheritance)،

3. `super()` بهتر از استفاده مستقیم از نام کلاس عمل می‌کنه چون ترتیب

4. MRO (Method Resolution Order) رو رعایت می‌کنه.

5. استفاده از متدهای کلاس والد رو می‌ده، بدون نیاز به تکرار نام کلاس.

6. بسیار مفید در سازنده‌ها (`__init__`) و برای گسترش متدهای والد.

7. در وراثت چندگانه، `super()` دقیق‌تر و ایمن‌تر از ارجاع مستقیم عمل می‌کنه.

تمرین

کلاسی به نام `Employee` تعریف کن که ویژگی `name` و متدی به نام `get_info()` داشته باشه.

کلاس دیگری به نام `Manager` بساز که از `Employee` ارث‌بری کنه، ویژگی `department` داشته باشه و متد `get_info()` رو `override` کنه ولی همچنان از نسخه والد استفاده کنه (با `super()`).

چندریختی (Polymorphism) یعنی:

یک تابع، متد یا عملگر، رفتار متفاوتی با توجه به نوع شیئی که روی آن اعمال می‌شود داشته باشد.

یعنی شما می‌تونید یک متد با همان نام داشته باشید که در کلاس‌های مختلف، رفتار متفاوتی دارد.

نکته مهم: Duck Typing در پایتون

پایتون یک زبان داینامیک است. یعنی وقتی می‌گوییم:

```
class Cat:
    def speak(self):
        print("Meow!")
```

```
class Dog:
    def speak(self):
        print("Woof!")
```

```
python
CopyEdit
: def do_something(x)
    ()x.speak
```

پایتون چک نمی‌کند که **x** چه کلاسی دارد؛ فقط باید متدی به اسم **speak()** داشته باشد.

```
def animal_sound(animal):
    animal.speak()
```

♦ به این می‌گویند Duck Typing:

"اگه مثل اردک راه بره و مثل اردک صدا بده، احتمالاً یه اردکه!" 🦆

تمرین

کلاس پایه‌ای **Employee** تعریف کن که متد **get_salary()** داشته باشد.
سپس دو کلاس **Manager** و **Developer** تعریف کن که از **Employee** ارث‌بری کنند و
متد **get_salary()** رو متفاوت پیاده‌سازی کنند.
سپس لیستی از این اشیاء بساز و همه حقوق‌ها را چاپ کن.

کلاس انتزاعی چیست؟

کلاس انتزاعی یک کلاس پایه است که فقط طرح کلی (interface) را تعریف می‌کند، ولی پیاده‌سازی کامل ندارد.

یعنی: نمی‌خوایم کسی از این کلاس شیء بسازد! بلکه فقط کلاس‌های فرزند باید ازش ارث‌بری کنند و متدهای تعریف‌شده رو پیاده‌سازی (implement) کنند.

abc ابزار: ماژول

:استفاده می‌کنیم **abc** در پایتون برای ساخت کلاس‌های انتزاعی از ماژول

```
from abc import ABC, abstractmethod
```

- **ABC**: پایه کلاس‌های انتزاعی
- **@abstractmethod**: دکوراتور برای تعریف متدهایی که باید توسط فرزندان پیاده‌سازی شوند

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):  
    @abstractmethod  
    def area(self):  
        pass
```

```
class Circle(Shape):  
    def area(self):  
        return 3.14 * (5 ** 2)
```

```
class Square(Shape):  
    def area(self):  
        return 4 * 4
```

```
shapes = [Circle(), Square()]  
for s in shapes:  
    print(s.area())
```

اگر متد **area** () در کلاس فرزند حذف شه:

```
class BadShape(Shape):  
    pass
```

**TypeError: Can't instantiate
abstract class BadShape with
abstract methods area**

تمرین

کلاسی به نام **Vehicle** بساز که انتزاعی باشد و متد انتزاعی **start_engine()** داشته باشد.

سپس دو کلاس **Car** و **Motorcycle** تعریف کن که این متد رو پیاده‌سازی کنن.
در نهایت، با حلقه‌ای از هر کدام شیء بساز و متد **start_engine()** رو صدا بزن.

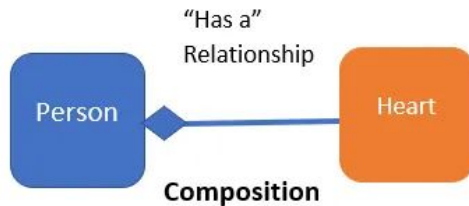
ترکیب یعنی چی؟ Composition

ترکیب یعنی: یک کلاس می‌تونه دارای شیء از کلاس دیگر باشه، به‌جای اینکه از اون ارث‌بری کنه. یعنی به‌جای اینکه بگیم "X یک Y است"، می‌گیم "X دارای Y است".

UML Diagram

مثلاً:

Super Class (Parent)



- یک ماشین دارای یک موتور است (Car has a Engine)

- یک کاربر دارای یک آدرس است (User has a Address)

Sub Class (Child)



Inheritance

Aggregation

⚖️ مقایسه با ارث‌بری

ویژگی	ارث‌بری	ترکیب
رابطه	is-a (هست)	has-a (داره)
انعطاف‌پذیری	کمتر	بیشتر
وابستگی	زیاد	کم
تغییرپذیری	سخت	راحت

چرا ترکیب بهتره؟

مزیت

کاهش وابستگی‌ها

قابلیت تست بالا

افزایش انعطاف

طراحی مدرن‌تر

توضیح

کلاس‌ها مستقل از هم می‌مونن و فقط همکاری می‌کنن

چون اجزا جدا هستن، راحت‌تر تست می‌شن

می‌تونن توی کلاس `Car` هر وقت خواستی موتور رو عوض کنی

ترکیب توی معماری‌های جدید مثل Clean Architecture، بسیار پرکاربردتره

```
class Engine:  
    def start(self):  
        print("Engine started")
```

```
class Car:  
    def __init__(self):  
        self.engine = Engine() #
```

تركيب

```
    def start(self):  
        print("Starting the car...")  
        self.engine.start()
```

```
c = Car()  
c.start()
```

تمرین

کلاسی بساز به نام **Battery** با متدی به نام **charge ()**.
سپس کلاسی بساز به نام **Phone** که دارای یک **Battery** باشد و متدی بنویس
به نام **power_on ()** که هنگام اجرا باتری رو شارژ کنه و گوشی روشن شه.

متد جادویی چیست؟

متدهای جادویی، متدهایی هستند که:

- نامشان با `__` شروع و پایان می‌یابد (مثل `__init__`, `__len__`, `__str__`, ...)

- توسط پایتون به‌طور خودکار فراخوانی می‌شوند

- برای کنترل رفتارهای داخلی و اپراتورها استفاده می‌شوند

به همین دلیل به آن‌ها می‌گن جادویی (magic) یا **Dunder =**

Double Underscore

کاربرد	متد
سازنده کلاس (همون که قبلاً یاد گرفتیم)	<code>__init__</code>
تعریف خروجی متنی کلاس هنگام <code>print()</code>	<code>__str__</code>
نمایش رسمی برای دیباگ	<code>__repr__</code>
تعیین طول با تابع <code>len()</code>	<code>__len__</code>
رفتار اپراتور <code>+</code>	<code>__add__</code>
مقایسه با <code>==</code>	<code>__eq__</code>
مقایسه کوچک‌تر/بزرگ‌تر از	<code>__lt__</code> , <code>__gt__</code>
رفتار کلاس مثل لیست (مثل <code>x[0]</code>)	<code>getitem__</code> , <code>__setitem__</code>

```
class Book:  
    def __init__(self, title,  
pages):  
        self.title = title  
        self.pages = pages  
  
    def __str__(self):  
        return f"Book:  
{self.title}"  
  
    def __len__(self):  
        return self.pages  
  
b = Book("Python Basics",  
300)  
  
print(b)      # Book: Python  
Basics  
print(len(b)) # 300
```

@property

ابزاری برای نوشتن کدی تمیزتر، امن‌تر و قابل نگهداری‌تر.

@property

```
def balance(self):  
    return self._balance
```

@balance.setter

```
def balance(self, amount):  
    if amount < 0:  
        raise ValueError("Balance can't be negative!")  
    self._balance = amount
```

```
acc = BankAccount(1000)  
print(acc.balance) # 1000  
acc.balance = 500   # OK  
acc.balance = -200  # ❌  
ValueError
```

- جلوی مقداردهی اشتباه رو می‌گیره

- اجازه می‌ده پشت یک ویژگی ساده، منطق پیچیده بنویسی

- از بیرون مثل ویژگی دیده می‌شه، ولی از داخل مثل متد عمل می‌کنه

Encapsulation

کپسوله‌سازی یعنی مخفی کردن جزئیات پیاده‌سازی یک شی و فراهم کردن فقط آن چیزی که برای استفاده کاربر لازم است.

یعنی:

- ویژگی‌ها و متدهایی که نباید از بیرون تغییر داده بشن → مخفی می‌شن
- فقط به رابط امن و کنترل‌شده در اختیار کاربر قرار می‌گیره

عمومی `Public`

محافظت شده `_Public`

خصوصی `__Public`

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # ← private
```

```
def deposit(self, amount):
    if amount > 0:
        self.__balance += amount
```

```
def withdraw(self, amount):
    if 0 < amount <= self.__balance:
        self.__balance -= amount
    else:
        print("Insufficient funds!")
```

```
def get_balance(self):
    return self.__balance
```

```
acc = BankAccount(1000)
acc.deposit(500)
acc.withdraw(300)
print(acc.get_balance()) # خروجی: 1200
```

```
acc.__balance = -99999 # ❌ این مقدار تغییر
# هست private نمی‌کند چون
print(acc.get_balance()) # هنوز هم: 1200
```

`print(acc.__balance)` # ❌ `AttributeError`
`print(acc._BankAccount__balance)` # ✅ پایتون فقط اسم رو تغییر داده

✅ کپسوله‌سازی یعنی محافظت از داده‌ها و محدود کردن دسترسی مستقیم به آن‌ها.

اهمیت کپسوله سازی

مزیت	توضیح
امنیت بیشتر	جلو تغییر داده‌های حساس گرفته می‌شه
نگهداری آسان‌تر	پیااده‌سازی می‌تونه عوض شه بدون اینکه کاربران متوجه بشن
جلوگیری از خطا	با اعتبارسنجی داخلی
افزایش خوانایی	چون رابط مشخصی برای کاربر طراحی می‌کنی